

Q Optimizer Init

We streamlined the overall structure of the notebook by removing unnecessary imports, consolidating repeated logic into helper functions, and reorganizing the imports and helper functions to appear at the beginning of the file for better readability. Type hints and detailed docstrings were added across all major functions through the use of ChatGPT, improving clarity, IDE integration, and ease of maintenance. Model parameters were also renamed to be more descriptive and aligned with their corresponding problem sections, making the code more self-explanatory. Additionally, we synthesized plotting logic into a few helper functions with the help of an LLM and modularized solver training by defining separate functions for each question, which encapsulate the setup, training, and evaluation steps in a clear and reusable way.

For plotting, we significantly improved modularity and consistency. The seed-aggregation logic was extracted into a dedicated helper function, `_plot_seed_aggregate(ax, ...)`, which computes the mean, minimum, and maximum across multiple seeds and visualizes the results with a shaded confidence band. The higher-level `plot_metrics(...)` API was redesigned to unify all plotting tasks, handling single or multiple solvers, diverse data extractors, automatic subplot creation, legend placement, and figure saving in one cohesive interface. It supports flexible extractor types, including attribute names (str), callable functions, and tuples for seed-aggregated plots. To ensure robustness and Pythonic design, the function validates input types, avoids in-place mutations, and leverages modern Matplotlib conventions like `plt.subplots()` with `axes.ravel()` for cleaner, more efficient layout handling. Additional customization options such as `figsize`, `marker`, `legend_loc`, and `save_path` make the visualization process both powerful and user-friendly.

We modularized the solver training process by defining a separate function for each experiment (e.g., **train_solver_2_1**, **train_solver_2_2**, etc.), making the codebase cleaner and easier to maintain. Each function now encapsulates its own logic for a given setup, including update rules, seeds, and batch sizes instead of relying on a single large script. We also made inputs explicit by passing parameters like `seeds`, `update_rules`, `batch_sizes`, and `small_data` directly into each function rather than referencing global variables. This makes the code more reproducible, testable, and flexible for future experiments. Finally, we added type hints and detailed NumPy-style docstrings, improving readability while clearly documenting parameters, side effects, and expected outputs for anyone extending or reusing the functions.